

Bookly — Team Work Breakdown (4 people)

Every optional feature is in scope, plus background jobs and Arabic/English localization. Work is split into 4 building blocks: yours is the largest by design (full MVP + the most integration-heavy pieces), and the remaining features are grouped into 3 blocks of equal effort for your teammates.

Block A — Leader (You)

Full MVP + Payments (Paymob) + Notifications + Messaging + Background Jobs

Included	Details
Auth	ASP.NET Identity, Roles/UserRoles, RefreshTokens, JWT-in-cookie flow
Listings	CRUD, photos, amenities
Search & Browse	Basic city/date/guest/price filtering
Booking flow	Request → confirm/reject → complete, overlap checking
Reviews	Base 1–5 rating + comment
Payments	Paymob order/checkout/webhook, <code>Payments</code> table
Notifications	Real-time via SignalR (email fallback), <code>Notifications</code> table
Messaging	Host↔guest inbox in real time via SignalR, <code>Messages</code> table
Background Jobs	Hangfire (or Quartz.NET): auto-complete bookings past checkout, retry failed Paymob webhooks, purge expired refresh tokens, batched notification digests

Entities owned: Users, Roles, UserRoles, RefreshTokens, Listings, ListingPhotos, Amenities, ListingAmenities, Bookings, Reviews (base), Payments, Notifications, Messages (*Hangfire manages its own job-storage tables automatically — nothing custom to design there*)

This block is the foundation everyone else builds on top of — get the schema and Auth

working first, everything downstream depends on it. Since Notifications and Messaging both run over SignalR, build them on one shared real-time setup rather than two unrelated systems.

Block B — Guest Trust

Wishlist/Favorites · Multi-category Ratings · Host Responses to Reviews · Cancellation Policies · Host Verification

Included	Details
Wishlist	Save/unsave a listing
Multi-category ratings	Cleanliness/Communication/Location/Value columns on Reviews
Host responses	One public reply per review
Cancellation policies	Flexible/Moderate/Strict enum shown on listing
Host verification	ID upload, verified badge, admin/auto approval

Entities owned: Wishlists, Reviews (rating columns — coordinate with Block A, don't duplicate the base table), HostResponses, Listings.CancellationPolicy, HostVerifications

Block C — Discovery & Promotions

Advanced Search · Availability Calendar · Coupons

Included	Details
Advanced search	Map view, amenity filters, instant-book toggle
Availability calendar	Host manually blocks/unblocks dates
Coupons	Discount codes, usage limits, expiry — surfaced in search results and applied at checkout

Entities owned: BlockedDates, Coupons. Advanced Search extends the existing `/Listings/Index` search — coordinate with Block A so you're extending it, not forking a second search endpoint.

Block D — Platform & Localization

Admin Panel · Localization (Arabic + English)

Included	Details
Admin panel	Dashboard stats, manage users, moderate listings
Localization	Full Arabic + English support — resource-based (<code>.resx</code>) translations, culture switching, RTL layout for Arabic

Entities owned: none new — Admin Panel reads existing tables, localization is resource-file based, not database-driven. Admin Panel is read-heavy across almost every table (Users, Listings, Bookings, Payments), so it's often easiest to start once the other blocks have real data to show.

Dependency Notes

- Block A's schema (Users/Roles/Listings/Bookings/Reviews) needs to land first — everyone else builds on it, so prioritize getting migrations in early even before every MVP view is polished.
- Blocks B and C can move in parallel with each other and with the back half of Block A.
- Block C's Coupons touch Payments/Bookings data — start that piece once Block A's Payments work has a working shape, not before.
- **Localization is cross-cutting, even though Block D owns the infrastructure:** every view any block writes needs its strings pulled from resource keys (`LocalizedString`) instead of hardcoded text, and layout needs to tolerate `dir="rtl"` (no hardcoded left/right margins — use logical CSS properties). Agree on the resource-key convention with Block D *before* the other three blocks start writing views, or it's a lot of rework later.
- Flag integration points early: Coupons (Block C) apply at Payments checkout (Block A); Notifications (Block A) should still fire on events from Blocks B/C (e.g. host verified, new host response) once those are ready — agree on a shared notification-trigger interface rather than each block bolting on its own.